

# Test Cases

Company Z – Employee Management System

Aligned to EmployeeDAO.java – Programming Tasks (a), (b), (c)

Each test case follows the format from the course's *Test Case Formatting Guide*: Input, Expected Output (Oracle), Initialization, Test Steps, Tear Down, and Dependency. The cases below target the actual public methods of `EmployeeDAO.java`. The `update`, `salary-raise`, and `report` methods all return `void` and report results to `System.out`, so those tests redirect `System.out` to a `ByteArrayOutputStream` during initialization and verify side effects through both captured `stdout` and follow-up SQL queries. `searchEmployee(String)` returns a `List<Employee>`, so its tests assert directly against the returned list rather than capturing `stdout`.

## Coverage Summary

| Programming Task           | DAO Method(s) Tested                                                                                       | Test Cases                                                 |
|----------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| (a) Update employee data   | <code>updateEmployee(int, String, String)</code><br><code>updateEmployeeFull(int, double, int, int)</code> | A1 (email/phone), A2 (full), A3 (missing ID)               |
| (b) Search for employee    | <code>searchEmployee(String) → List&lt;Employee&gt;</code>                                                 | B1 (found by name), B2 (not found → empty list)            |
| (c) Update salary by range | <code>increaseSalary(double, double, double)</code>                                                        | C1 (with boundaries), C2 ("less than X"), C3 (empty range) |

## Test Case A1: updateEmployee – Update Email and Phone (Happy Path)

Maps to programming task: (a) Update employee data – tests `EmployeeDAO.updateEmployee(int, String, String)`

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• <code>emplID = 1001</code></li><li>• <code>newEmail = "jane.doe@newdomain.com"</code></li><li>• <code>newPhone = "555-0199"</code></li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output contains the line "Employee updated."</li><li>• Row in employee table for <code>emplID=1001</code> has email = "jane.doe@newdomain.com" and phone = "555-0199"</li><li>• All other columns (<code>firstName</code>, <code>lastName</code>, <code>SSN</code>, <code>hireDate</code>, <code>salary</code>, <code>jobTitleID</code>, <code>divisionID</code>) are unchanged from baseline</li></ul>                                                                                                                                                                                                                                     |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• MySQL <code>employeeData</code> database is running and <code>DatabaseConnection.getConnection()</code> succeeds</li><li>• Row exists: <code>emplID=1001</code>, <code>firstName="Jane"</code>, <code>lastName="Doe"</code>, <code>SSN="123456789"</code>, <code>email="jane.doe@old.com"</code>, <code>phone="555-0100"</code>, <code>hireDate="2020-06-15"</code>, <code>salary=75000.00</code>, <code>jobTitleID=2</code>, <code>divisionID=1</code></li><li>• Capture baseline row into a local map for later comparison</li><li>• Redirect <code>System.out</code> to a <code>ByteArrayOutputStream</code> to capture console output</li></ul> |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate <code>EmployeeDAO dao = new EmployeeDAO();</code></li><li>2. Call <code>dao.updateEmployee(1001, "jane.doe@newdomain.com", "555-0199");</code></li><li>3. Assert captured stdout contains the substring "Employee updated."</li><li>4. Execute <code>SELECT * FROM employee WHERE emplID = 1001</code></li><li>5. Assert email == "jane.doe@newdomain.com" and phone == "555-0199"</li><li>6. Assert <code>firstName</code>, <code>lastName</code>, <code>SSN</code>, <code>hireDate</code>, <code>salary</code>, <code>jobTitleID</code>, <code>divisionID</code> all match the captured baseline</li></ol>                           |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore <code>emplID=1001</code> row to baseline values</li><li>• Restore <code>System.out</code> to its original <code>PrintStream</code></li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

## Test Case A2: updateEmployeeFull – Update Salary, Job Title, and Division (Happy Path)

Maps to programming task: (a) Update employee data – tests `EmployeeDAO.updateEmployeeFull(int, double, int, int)`

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• empID = 1001</li><li>• salary = 92000.00</li><li>• jobTitleID = 5</li><li>• divisionID = 3</li></ul>                                                                                                                                                                                                                                                                                                                  |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output contains the line "Employee updated (full)."</li><li>• Row for empID=1001 has salary=92000.00, jobTitleID=5, divisionID=3</li><li>• All other columns (firstName, lastName, SSN, email, phone, hireDate) are unchanged</li></ul>                                                                                                                                                                       |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Row exists for empID=1001 (same baseline as A1)</li><li>• Reference rows exist in jobTitle table for jobTitleID=5 and in division table for divisionID=3 (foreign-key constraints will fail otherwise)</li><li>• Capture baseline row into a local map</li><li>• Redirect System.out to a ByteArrayOutputStream</li></ul>                                                      |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate EmployeeDAO dao = new EmployeeDAO();</li><li>2. Call dao.updateEmployeeFull(1001, 92000.00, 5, 3);</li><li>3. Assert captured stdout contains "Employee updated (full)."</li><li>4. Execute SELECT * FROM employee WHERE empID = 1001</li><li>5. Assert salary == 92000.00, jobTitleID == 5, divisionID == 3</li><li>6. Assert firstName, lastName, SSN, email, phone, hireDate match baseline</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore empID=1001 row to baseline values</li><li>• Restore System.out</li></ul>                                                                                                                                                                                                                                                                                                                                      |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                        |

## Test Case A3: updateEmployee – Non-Existent EmpID (Edge Case)

Maps to programming task: (a) Update employee data – exposes that updateEmployee does not check rowsAffected

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• empID = 9999 (guaranteed not present in employee table)</li><li>• newEmail = "ghost@example.com"</li><li>• newPhone = "555-0000"</li></ul>                                                                                                                                                                                                                                                                                                                     |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output still contains "Employee updated." (current implementation prints this regardless of rows affected)</li><li>• No row is created or modified – SELECT COUNT(*) FROM employee WHERE empID = 9999 still returns 0</li><li>• No row in employee table has email = "ghost@example.com" or phone = "555-0000"</li></ul>                                                                                                                               |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Confirm SELECT COUNT(*) FROM employee WHERE empID = 9999 returns 0</li><li>• Snapshot full employee table (e.g., dump to a temp table or capture row count + checksum)</li><li>• Redirect System.out to a ByteArrayOutputStream</li></ul>                                                                                                                                                                               |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate EmployeeDAO dao = new EmployeeDAO();</li><li>2. Call dao.updateEmployee(9999, "ghost@example.com", "555-0000");</li><li>3. Assert captured stdout contains "Employee updated."</li><li>4. Re-confirm SELECT COUNT(*) FROM employee WHERE empID = 9999 == 0</li><li>5. Assert SELECT COUNT(*) FROM employee WHERE email = "ghost@example.com" == 0</li><li>6. Compare full-table snapshot to current state and assert they are identical</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore System.out</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

**Note:** Documents a known weakness: the DAO prints the success message even when zero rows match. A future hardening task should change updateEmployee to inspect ps.executeUpdate() and print a different message when 0 rows are affected.

## Test Case B1: searchEmployee – Found by Name (Happy Path)

Maps to programming task: (b) Search for employee – tests `EmployeeDAO.searchEmployee(String)`

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• searchTerm = "Jane Doe" (string passed as name; the DAO accepts any of name, SSN, or empID-as-string)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Returned List&lt;Employee&gt; is non-null</li><li>• Returned list contains at least one Employee</li><li>• Exactly one Employee in the list has empID == 1001</li><li>• That Employee has firstName == "Jane", lastName == "Doe", SSN == "123456789"</li><li>• That Employee's salary, hireDate, jobTitleID, and divisionID match the baseline</li><li>• If the Employee object exposes joined fields, divisionName and jobTitleName are populated (e.g., "Engineering" and "Software Engineer")</li></ul>                                                                                                                                             |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Row exists: empID=1001, firstName="Jane", lastName="Doe", SSN="123456789", salary=75000.00, jobTitleID=2, divisionID=1</li><li>• Reference data exists: division row with divisionID=1 and divisionName="Engineering"; jobTitle row with jobTitleID=2 and jobTitleName="Software Engineer"</li><li>• No second employee in the table also has firstName="Jane" and lastName="Doe" (otherwise the "exactly one" assertion below must be relaxed)</li></ul>                                                                                                                                                       |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate EmployeeDAO dao = new EmployeeDAO();</li><li>2. Call List&lt;Employee&gt; results = dao.searchEmployee("Jane Doe");</li><li>3. Assert results != null</li><li>4. Assert !results.isEmpty()</li><li>5. Filter results to those with empID == 1001 and assert exactly one match; call it found</li><li>6. Assert found.getFirstName().equals("Jane")</li><li>7. Assert found.getLastName().equals("Doe")</li><li>8. Assert found.getSsn().equals("123456789")</li><li>9. Assert found.getSalary() == 75000.00 (within a small delta if using double)</li><li>10. Assert found.getJobTitleID() == 2 and found.getDivisionID() == 1</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• None (no state changed)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• Depends on the empID=1001 baseline row and on division / jobTitle reference data — shares preconditions with Test Case A1</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

## Test Case B2: searchEmployee – Not Found Returns Empty List (Negative Path)

Maps to programming task: (b) Search for employee – tests the not-found branch of searchEmployee(String)

|                                 |                                                                                                                                                                                                                                                                                                                     |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• searchTerm = "9999" (no employee has this empID, name, or SSN)</li></ul>                                                                                                                                                                                                    |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Returned List&lt;Employee&gt; is non-null</li><li>• Returned list is empty (size == 0)</li><li>• No exception is thrown</li></ul>                                                                                                                                           |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Confirm SELECT COUNT(*) FROM employee WHERE empID = 9999 returns 0</li><li>• Confirm no employee row has SSN = "9999"</li><li>• Confirm no employee row has firstName, lastName, or firstName+" "+lastName equal to "9999"</li></ul> |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate EmployeeDAO dao = new EmployeeDAO();</li><li>2. Call List&lt;Employee&gt; results = dao.searchEmployee("9999");</li><li>3. Assert no exception was thrown</li><li>4. Assert results != null</li><li>5. Assert results.isEmpty()</li></ol>                      |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                              |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                              |

## Test Case C1: increaseSalary – Range Raise with Boundary Verification (Happy Path)

Maps to programming task: (c) Update salary for employees in a range – tests `EmployeeDAO.increaseSalary(double, double, double)`

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• min = 58000.00 (inclusive – salary <math>\geq</math> 58000)</li><li>• max = 105000.00 (exclusive – salary <math>&lt;</math> 105000)</li><li>• percent = 0.032 (i.e., 3.2%)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output contains "&lt;N&gt; employees updated." where N is the count of employees whose pre-call salary satisfied <math>58000 \leq \text{salary} &lt; 105000</math></li><li>• Every employee whose pre-call salary was in [58000, 105000) now has <code>newSalary == round(oldSalary <math>\times</math> 1.032, 2)</code></li><li>• Every employee whose pre-call salary was outside that range has unchanged salary</li><li>• Boundary: employee with pre-call salary == 58000.00 IS updated to 59856.00</li><li>• Boundary: employee with pre-call salary == 105000.00 is NOT updated</li></ul>                                                                                                                                                                                                             |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Seed (or confirm) employees with salaries spanning the boundaries: 57999.99, 58000.00, 75000.00, 104999.99, 105000.00, 120000.00</li><li>• Capture every (empID, salary) pair into a <code>Map&lt;Integer, Double&gt;</code> called <code>preCall</code></li><li>• Compute expected N = <code>SELECT COUNT(*) FROM employee WHERE salary <math>\geq</math> 58000 AND salary <math>&lt;</math> 105000</code></li><li>• Redirect <code>System.out</code> to a <code>ByteArrayOutputStream</code></li></ul>                                                                                                                                                                                                                                                                      |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate <code>EmployeeDAO dao = new EmployeeDAO();</code></li><li>2. Call <code>dao.increaseSalary(58000.00, 105000.00, 0.032);</code></li><li>3. Assert captured stdout contains the substring <code>N + " employees updated."</code></li><li>4. Execute <code>SELECT empID, salary FROM employee</code> and load into <code>postCall</code> map</li><li>5. For each <code>empID</code> in <code>preCall</code>: if <code>old <math>\in</math> [58000, 105000)</code> assert <code>postCall[empID] == round(old <math>\times</math> 1.032, 2)</code>; else assert <code>postCall[empID] == old</code></li><li>6. Assert <code>postCall[employee with old=58000.00] == 59856.00</code> (boundary in)</li><li>7. Assert <code>postCall[employee with old=105000.00] == 105000.00</code> (boundary out)</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore every employee's salary from the <code>preCall</code> map</li><li>• Restore <code>System.out</code></li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## Test Case C2: increaseSalary – "Less Than X" Threshold Use Case (Happy Path)

Maps to programming task: (c) Implements the user-story phrase "update salary for all employees less than a particular amount" by passing min = 0

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• min = 0.00</li><li>• max = 60000.00 (exclusive – "less than 60K")</li><li>• percent = 0.05 (5%)</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output contains "&lt;N&gt; employees updated." where N is the count of employees whose pre-call salary satisfied <math>0 \leq \text{salary} &lt; 60000</math></li><li>• Every employee with pre-call salary <math>&lt; 60000</math> now has <code>newSalary == round(oldSalary × 1.05, 2)</code></li><li>• Every employee with pre-call salary <math>\geq 60000</math> has unchanged salary</li></ul>                                                                                                                   |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Seed (or confirm) at least these salaries exist: 35000.00, 50000.00, 59999.99, 60000.00, 80000.00</li><li>• Capture (empID, salary) into preCall map</li><li>• Compute <code>N = SELECT COUNT(*) FROM employee WHERE salary &lt; 60000</code></li><li>• Redirect System.out</li></ul>                                                                                                                                                                                                    |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate <code>EmployeeDAO dao = new EmployeeDAO();</code></li><li>2. Call <code>dao.increaseSalary(0.00, 60000.00, 0.05);</code></li><li>3. Assert captured stdout contains <code>N + " employees updated."</code></li><li>4. Load postCall salaries from DB</li><li>5. For each empID: if <code>old &lt; 60000</code> assert <code>postCall == round(old × 1.05, 2)</code>; else assert <code>postCall == old</code></li><li>6. Specifically assert the employee with <code>old=60000.00</code> was NOT updated</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore every employee's salary from preCall</li><li>• Restore System.out</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## Test Case C3: increaseSalary – Empty Range Matches No Employees (Edge Case)

Maps to programming task: (c) Verifies the no-op path of increaseSalary when no employees fall in the range

|                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>                    | <ul style="list-style-type: none"><li>• min = 999000.00</li><li>• max = 1000000.00</li><li>• percent = 0.10</li></ul>                                                                                                                                                                                                                                                                                                                |
| <b>Expected Output (Oracle)</b> | <ul style="list-style-type: none"><li>• Console output contains "0 employees updated."</li><li>• No salary in the employee table changes</li><li>• No exception is thrown</li></ul>                                                                                                                                                                                                                                                  |
| <b>Initialization</b>           | <ul style="list-style-type: none"><li>• Database connection verified</li><li>• Confirm <code>SELECT COUNT(*) FROM employee WHERE salary &gt;= 999000 AND salary &lt; 1000000 == 0</code></li><li>• Capture every (empID, salary) into preCall map</li><li>• Redirect System.out</li></ul>                                                                                                                                            |
| <b>Test Steps</b>               | <ol style="list-style-type: none"><li>1. Instantiate EmployeeDAO <code>dao = new EmployeeDAO();</code></li><li>2. Call <code>dao.increaseSalary(999000.00, 1000000.00, 0.10);</code></li><li>3. Assert no exception thrown</li><li>4. Assert captured stdout contains "0 employees updated."</li><li>5. Load postCall salaries from DB</li><li>6. Assert <code>preCall.equals(postCall)</code> – every salary is identical</li></ol> |
| <b>Tear Down</b>                | <ul style="list-style-type: none"><li>• Restore System.out</li></ul>                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Dependency</b>               | <ul style="list-style-type: none"><li>• None</li></ul>                                                                                                                                                                                                                                                                                                                                                                               |